

Wishtrip AI Engineering Internship Assignment

Build a personalised trip-planning prototype

Build a working full-stack prototype that demonstrates how you would turn a traveller's preferences into a practical, personalised, day-by-day itinerary using a combination of data, software engineering, and AI or algorithmic decision-making.

The challenge

Build a web application that collects trip preferences and generates a day-by-day itinerary using a travel database created or sourced by you.

Your planner should collect at least:

- destination and origin city;
- travel dates or month, and number of nights;
- traveller type and party composition (for example solo, couple, family, friends, or seniors);
- interests (for example food, culture, nature, beaches, wellness, or adventure);
- preferred pace (easy-going, balanced, or packed); and
- comfort or budget level.

You may add inputs such as dietary preferences, mobility needs, or preferred transport if they improve the itinerary meaningfully.

Expected output

Return a clear itinerary for every calendar day. Each day should include selected activities or places, a sensible order, indicative timings or parts of day, and enough explanation for the traveller to understand why the plan suits their preferences. Include travel time, meals, stays, or indicative costs where your data supports them.

We are looking for reasonable trip quality, not just a polished interface or an LLM-generated list. A good itinerary should respect trip length, geography and travel time, opening hours where available, activity duration, pace, party suitability, budget, and duplicate or conflicting activities. When reliable information is unavailable, state the assumption instead of presenting it as fact.

What to build

Your submission must be a working full-stack prototype with:

- a usable frontend for collecting inputs and viewing the itinerary;
- a backend API that accepts a structured trip request and returns a structured itinerary;
- a database or clearly defined data layer containing the places and attributes used by the planner; and
- a planning component - rules, optimisation, ranking, an LLM, a traditional ML model, or a thoughtful combination - that converts the request and data into the final plan.

Scope and data

You do not need to support every destination. One destination with enough quality data to demonstrate varied, credible itineraries is better than broad but shallow coverage. You may use public or synthetic data, but identify its source and do not use Wishtrip's private code, APIs, or datasets.

Submission

Please provide:

1. A link to the working prototype and a source-code repository with setup instructions.
2. A short README explaining the architecture, technology choices, data model and sources, planning approach, constraints, assumptions, and known limitations.
3. Two or three example trip requests showing how different inputs produce meaningfully different itineraries.

The project should run from a clean setup. Automated tests are not mandatory, but focused tests for important planning rules will strengthen the submission. Be prepared to walk us through the code and explain what you would improve with more time and better data.

How we will evaluate it

- **Itinerary quality and personalisation** - Is the plan relevant, feasible, coherent, and visibly shaped by the traveller's inputs?
- **Planning intelligence and constraint handling** - Is the approach more than prompt wrapping? Are constraints, trade-offs, uncertainty, and failure cases handled thoughtfully?
- **Engineering quality** - Is the frontend-backend-data flow complete, reliable, understandable, and reasonably tested?
- **Product experience** - Is the application easy to use and is the output useful to a traveller?
- **Communication and judgment** - Can the candidate clearly explain the technologies used, choices made, alternatives considered, limitations, and next steps?

Use any technology stack you can explain and defend. We value sound judgment, honest trade-offs, and a smaller system that works well over unnecessary complexity.